Rapport Robot de Réconfort

Thomas Breton, Ibraïma Dramé

L'objectif de ce projet est la réalisation d'un agent (robot) capable d'évoluer dans un environnement (un appartement) afin de répondre à des demandes faites par les différents résidents. Au départ, le robot n'a aucune indication sur ce qui l'entoure et sur le contenu de l'armoire, armoire contenant les objets pour répondre aux besoins des résidents. Le robot doit ainsi pouvoir se baser sur ses propres observations afin de pouvoir accomplir sa tâche du mieux possible.

1) Architecture (Perception, mémoire, décision, action)

L'architecture du code est basée sur trois classes principales, Agent, Armoire, Dictionnaire, un fichier main afin de gérer tout cela et une classe Trace consignant les actions faites.

La classe Agent est en quelque sorte le chef d'orchestre du robot, elle comporte plusieurs méthodes utiles à son fonctionnement. Cette classe permet entre autres de centraliser l'état du robot, ce qui facilite la consultation et l'actualisation des données lors de la simulation. Les attributs de cette classe représentent les informations que le robot a en sa possession, en quelque sorte, ses attributs sont sa Mémoire. Ses attributs représentent en effet les positions importantes des différents acteurs du scénario (armoire, dictionnaire, résidents) ainsi que sa position et sa carte mentale nécessaire pour ses déplacements. Il y figure également des attributs permettant de voir l'état du robot, nous permettant de surveiller la cohérence entre ses états et ses actions (action, demande, main...). Les méthodes de la classe Agent permettent une partition propre du projet, chacune d'entre elles possède un rôle simple. Les méthodes d'action (consulter, chercher…) sont à cheval entre la partie Action et Mémoire, l'idée est de venir actualiser la mémoire du robot (donc ses attributs) après qu'une action ait été effectuée, main du robot, sa position, sa carte… Dans le même principe, la méthode cartographie constitue la Perception du robot, mais elle vient tout de suite mettre à jour la carte du robot et donc modifie sa Mémoire, là où la méthode perception ne fait que renvoyer les alentours du robot à un instant donné.

La classe Armoire est un attribut de la classe Agent, elle représente en quelque sorte les informations concernant l'armoire et fait donc partie en elle-même de la partie Mémoire. On peut imaginer que le robot vient lire ses informations une fois qu'il a atteint l'armoire, cette classe nous permet ainsi de séparer la gestion du robot de celle de l'armoire, ce qui nous permet de gérer la recherche et le repli indépendamment de l'agent. Ici encore, la classe fait appel à plusieurs petites méthodes telles que moveCursor qui bouge le sélecteur selon une direction donnée (Action). createPath ainsi que findCandidate jouent le rôle de décideurs puisque ces fonctions décident respectivement du chemin que le sélecteur doit suivre et de l'ordre des casiers visités en cas de repli. Au fur et à mesure des déplacements, updateArmoire vient mettre à jour la carte de l'armoire et donc actualise la Mémoire. Pour finir, la fonction fouillerCasier fait appel à toutes les autres afin d'organiser la recherche d'un casier, elle est donc à cheval dans toutes les catégories.

Dans la classe Dictionnaire, nous venons simplement trouver l'émotion ainsi que l'intensité liées à la demande du résident et rien d'autre. Cette classe est donc essentielle puisqu'elle détermine le casier cible, donc influe sur les décisions. Et finalement, dans le main, ces trois classes sont utilisées afin d'implémenter le robot. Son rôle est de lancer les actions adéquates en fonction de l'état du robot. Cela relève donc de la prise de décision. L'avantage est qu'elle permet également la séparation entre l'implémentation des actions et la partie simulation.

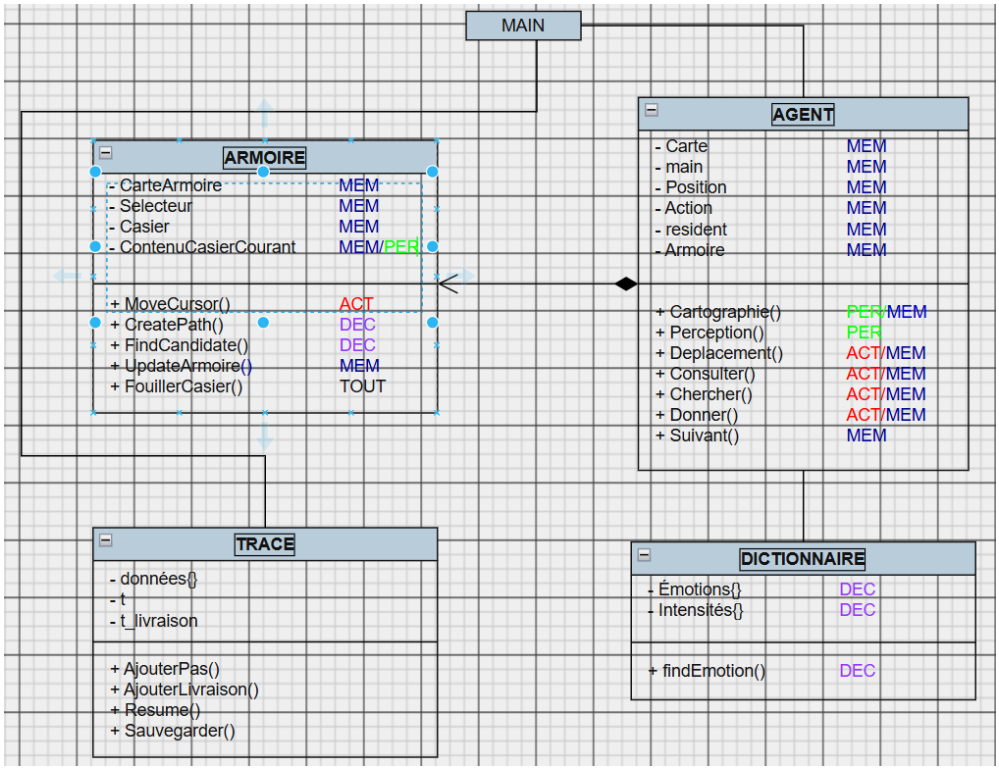


Diagramme des classes + Exposition des frontières

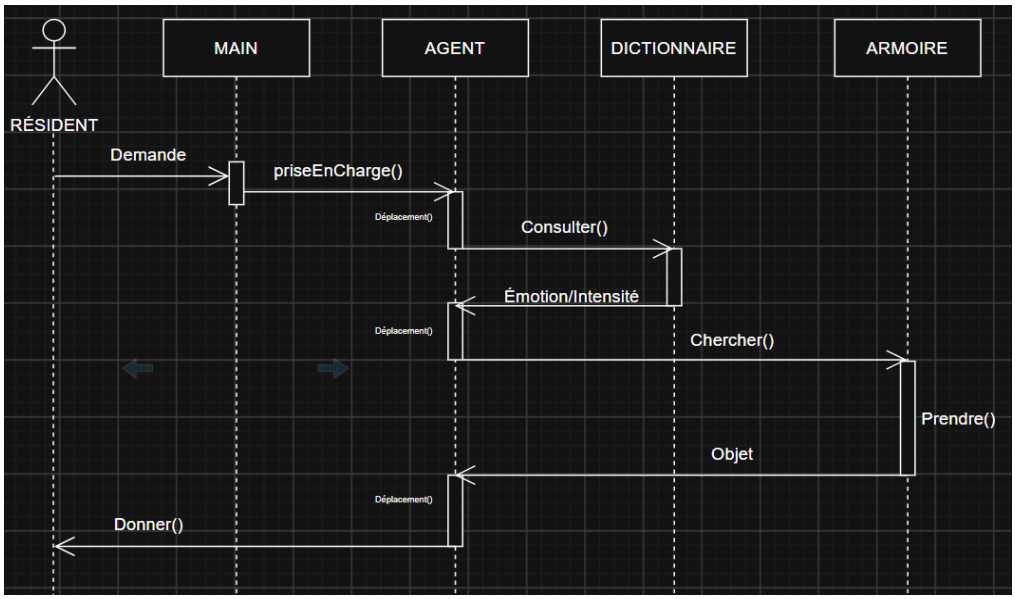


Diagramme de séquence (une seule demande)

2) Replanification

Pour la recherche du chemin, l'agent part du principe qu'il n'y a aucun mur et cherche ainsi le plus court chemin entre sa position actuelle et sa destination. Il actualise sa carte mentale au fur et à mesure de ses pas afin de savoir si une case présente un accès libre ou non. La recherche du plus court chemin garantit un coût minimum de déplacements, en sachant que peu importe la direction, le coût du pas est égal. Lorsque l'agent rencontre un mur, celui-ci recalcule simplement le plus court chemin après avoir remis sa carte mentale à jour. Il replanifie donc tout son chemin, ce qui est plus simple d'un point de vue implémentation, comme la carte mentale a été actualisée. Ainsi, le calcul de chemin se fait plusieurs fois lors de la simulation et entraîne des calculs que l'on peut juger inutiles, le coût peut donc être élevé même pour un déplacement d'une seule case.

3) Stratégie de repli

Pour la stratégie de repli, l'agent se base sur la roue de Plutchik, cette méthode permet de choisir un casier à une distance inférieure à 2 sur cette même roue. Cette stratégie ici est simple, déterministe et facilement implémentable. Néanmoins, elle risque de renvoyer un objet qui, bien qu'étant proche de l'émotion cible, ne soit pas adapté à la demande, par exemple, la joie et la colère sont voisines de distance 2, donc une demande interprétée comme joie peut être répondue par un objet de colère comme une balle antistress, ce qui est incohérent avec la demande. Une première idée ici serait de limiter le repli à une distance de 1 ou bien de le limiter à la même émotion d'intensités différentes.

Ainsi, comme nouvelle stratégie, on peut envisager d'associer aux objets des cas d'utilisation. Par exemple, on peut considérer que l'objet couverture, initialement prévu pour répondre à la tristesse, peut également être utilisé pour répondre à la peur (pour se cacher) ou pour apaiser une colère. Ainsi, on peut associer des émotions à un objet ou bien même des couples émotions-intensités. Cette nouvelle stratégie nécessitera donc l'ajout d'une liste d'émotions/intensités associées aux objets, le robot viendrait alors simplement boucler sur chaque objet pour voir si l'un d'eux peut également répondre au couple émotion-intensité initialement trouvé.

4) Tests

Prenons le cas où la carte mentale du robot n'est pas complète comme première situation. Pour aller d'une position à une autre, le robot calcule le plus court chemin en se basant sur sa carte mentale. On constate qu'il peut parfois se « tromper » de chemin et même faire demi-tour lors de son exploration. Ceci est dû au fait qu'il considère qu'une case est libre tant qu'il n'a pas eu confirmation de sa perception qu'elle ne l'est pas. On peut donc considérer que le robot prend des chemins inutiles et effectue des calculs inutiles, ce qui augmente le coût en déplacement.

Comme seconde situation, prenons le cas où l'armoire est vide. Notre agent possède en mémoire une « carte armoire », correspondant aux casiers qu'il a déjà visités. Bien que cette carte se mette à jour lors de la recherche dans le casier, on constate que le robot ne la prend pas en compte. En effet, lorsque le casier cible est vide, le robot enclenche sa stratégie de repli. Toutefois, lorsque l'armoire est totalement vide, le robot effectue tout de même ses recherches et se met donc à placer le curseur sur 15 casiers vides différents avant d'abandonner la recherche. Cela est dû au fait qu'il ne prenne pas l'initiative de ne plus visiter l'armoire, alors qu'il la sait vide, mais aussi au fait qu'il ne fasse aucune action spécifique lorsqu'il sait qu'un casier est vide.

Finalement, le cas où un résident est inaccessible met en avant un autre problème. Dans cette situation, le robot cherchera le chemin vers un résident, constatera qu'il est inaccessible et donc passera à la requête suivante. Cependant, le robot ne le constatera qu'une fois qu'il aura effectué toutes les tâches préalables comme prendre la demande, consulter, chercher… Le problème ici est qu'il ne vérifie pas en amont si toutes les conditions sont réunies pour accomplir sa tâche. On ajoute à cela le fait qu'il n'actualise pas les informations concernant les résidents. Ainsi, un résident inaccessible ayant effectué plusieurs demandes lancera le robot dans son exécution plusieurs fois, bien que le robot ait déjà établi qu'il n'est pas accessible. On a donc ici des actions inutiles, qui augmentent le coût, mais aussi « gaspillent » les objets puisque le robot ne repose pas les objets pris.